МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ТАРАСА ШЕВЧЕНКА

ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ

Практична робота № 2

з дисципліни **«Технології створення програмних продуктів»**

Тема: «Архітектура IoT екосистеми»

Виконав:

студент гр. ІР-31

Петро Кравченко

Перевірив:

Зараховано від

Кравченко Ольга Віталіївна

_____

(підпис викладача)

Київ-2026

ЗМІСТ

1. МЕТА ТА ЗАВДАННЯ РОБОТИ 3
2. ТЕОРЕТИЧНІ ВІДОМОСТІ 4
2.1. IoT-екосистема та рівні архітектури IoT-рішення 4
2.2. Архітектурні стилі програмних систем 5
2.3. Шаблони проектування та принципи SOLID 6
3. ВИБІР РІВНІВ РЕАЛІЗАЦІЇ ПРОГРАМНОГО ПРОДУКТУ 7
3.1. Вихідні вимоги до архітектури 7
3.2. Вибір рівнів реалізації 7
3.3. Вибір архітектурного стилю 8
4. АРХІТЕКТУРА ІОТ-ЕКОСИСТЕМИ «ЩИТ-ЛІНК» 10
4.1. Загальна схема архітектури 10
4.2. Рівень датчиків (сприйняття) 12
4.3. Мережевий рівень 12
4.4. Рівень обробки даних 14
4.5. Рівень застосунків 17
4.6. Фізичне розміщення компонентів 17
4.7. Наскрізний потік даних і бюджет затримок 18
4.8. Безпека та надійність 20
4.9. Шаблони проектування та принципи SOLID в архітектурі 20
4.10. Відповідність архітектури вимогам 21
ВИСНОВКИ 23
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 24

1. МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи: вивчити багаторівневу архітектуру IoT-екосистеми та архітектурні стилі програмних систем, обрати рівні реалізації власного програмного продукту, побудувати й описати архітектуру IoT-рішення.

Завдання роботи [1]: побудувати архітектуру IoT-рішення, обравши рівні реалізації власного програмного продукту; описати архітектуру. Відповідно до завдання у роботі:

1) розглянуто еталонну архітектуру IoT-екосистеми та архітектурні стилі — розділ 2;

2) обрано рівні реалізації та архітектурний стиль програмного продукту з обґрунтуванням вимогами — розділ 3;

3) побудовано архітектуру IoT-рішення та описано кожен рівень, розміщення компонентів, потоки даних, безпеку і застосовані шаблони проектування — розділ 4;

4) сформульовано висновки — розділ «Висновки».

Тема програмного продукту: «Розподілена IoT-система цілевказівки та координації дронів для перехоплення повітряних цілей»; програмний продукт — IoT-платформа «Щит-Лінк», вимоги до якої визначено в практичній роботі № 1 [2]. Звіт оформлено відповідно до ДСТУ 3008:2015 [3].

Примітка: РЦПДО «Щит-Північ» і IoT-платформа «Щит-Лінк» є навчальними умовними моделями; усі назви, процеси та показники наведено суто з навчально-ілюстративною метою.

2. ТЕОРЕТИЧНІ ВІДОМОСТІ

2.1. IoT-екосистема та рівні архітектури IoT-рішення

IoT-екосистема поєднує фізичні об’єкти з датчиками, засоби зв’язку, платформу обробки даних та інтерфейси користувачів. У публікаціях кафедри архітектуру IoT-рішення подають чотирма рівнями (рис. 2.1) [4]. **Рівень сприйняття** (Perception Layer) збирає дані від датчиків, розміщених на об’єктах спостереження. **Мережевий рівень** (Network Layer) забезпечує швидку передачу даних далі — бездротовими мережами Wi-Fi, Bluetooth тощо. **Рівень проміжного програмного забезпечення** (Middleware Layer) використовує хмарне сховище для обробки та зберігання великих обсягів інформації й аналізу даних у реальному часі. **Рівень застосунків** (Application Layer) відображає оброблені дані та забезпечує взаємодію користувачів із ними на різних пристроях [4].

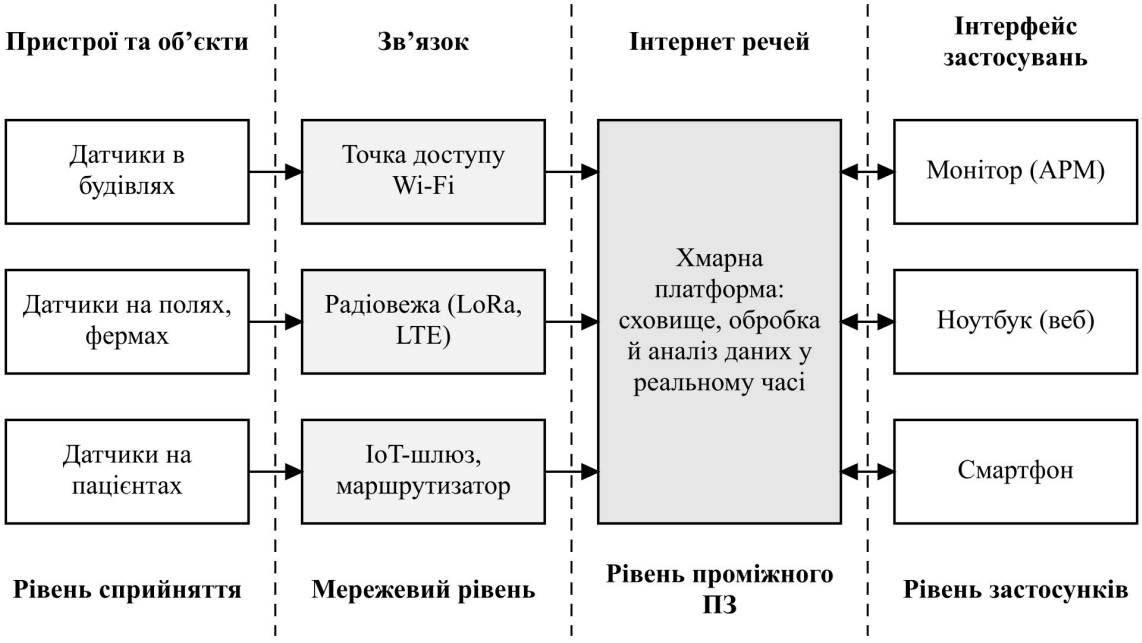


Рис. 2.1 — Узагальнена архітектура IoT-екосистеми (за [4])

Тези кафедри [5] використовують ту саму структуру з українськими назвами рівнів: рівень датчиків (Sensing Layer), мережевий рівень із комунікацією через Wi-Fi, MQTT, LoRa, Bluetooth та IoT-шлюзами для обробки й передачі даних у хмару, рівень обробки даних і рівень застосунків.

У статті про IoT-систему насосної станції [6] архітектуру також побудовано за схемою «пристрої — зв’язок — хмара — інтерфейс користувача»: дані датчиків проходять через бездротові мережі (Wi-Fi, Bluetooth, Zigbee, Z-Wave) та оптоволоконні медіаконвертери до бази даних, доступної оператору. Типовий склад рівнів узагальнено в табл. 2.1.

Таблиця 2.1 — Рівні архітектури IoT-рішення та їх типові компоненти

Рівень	Призначення	Типові компоненти
Рівень датчиків (сприйняття)	Збір даних про стан об’єктів і середовища	Датчики, вимірювальні модулі, мікроконтролери, виконавчі пристрої
Мережевий рівень	Передавання даних від пристроїв до платформи	Точки доступу Wi-Fi, LoRa, стільниковий зв’язок, IoT-шлюзи, протокол MQTT
Рівень обробки даних (проміжного ПЗ)	Зберігання, обробка та аналіз даних у реальному часі	Хмарні або власні сервери, бази даних, брокери повідомлень, сервіси аналітики
Рівень застосунків	Відображення даних і керування системою	Веб-застосунки, мобільні застосунки, панелі моніторингу, інтерфейси інших систем

2.2. Архітектурні стилі програмних систем

Проектування програмних систем — процес створення структури та архітектури ПЗ для забезпечення надійності, ефективності та масштабованості; одним з його етапів є вибір архітектурного стилю [7]. Лекція [7] розглядає чотири підходи:

– **монолітна архітектура** — усі функції (інтерфейс, логіка, доступ до бази даних) в одному додатку й одному процесі; простота розробки й розгортання на ранніх етапах, але складність масштабування та тісна зв’язність модулів;

– **мікросервісна архітектура** — набір незалежних сервісів, кожен з яких відповідає за конкретну функціональність і взаємодіє через API (REST, gRPC, черги повідомлень); незалежне масштабування й оновлення, стійкість до збоїв окремих сервісів, але складніші координація, тестування й мережеві витрати;

– **сервіс-орієнтована архітектура (SOA)** — сервіси бізнес-функцій зі стандартизованими інтерфейсами, придатні до повторного використання; на відміну від мікросервісів, використовує централізовану шину (ESB) і є більш крупнозернистою;

– **подієво-орієнтована архітектура (EDA)** — компоненти взаємодіють через обмін подіями: видавці подій, підписники та брокер подій (наприклад, Apache Kafka, RabbitMQ); асинхронність, відокремленість і обробка в реальному часі. Лекція прямо наводить Інтернет речей як приклад застосування EDA: пристрої IoT постійно генерують події, на які система реагує в реальному часі [7].

2.3. Шаблони проектування та принципи SOLID

Структурні шаблони проектування допомагають організувати класи й об’єкти у великі структури: «Адаптер» узгоджує несумісні інтерфейси; «Міст» розділяє абстракцію та реалізацію; «Компонувальник» будує ієрархії; «Декоратор» динамічно додає функціональність; «Фасад» надає спрощений інтерфейс до складної підсистеми; «Проксі» контролює доступ до об’єкта (кешування, перевірка прав) [7]. Серед поведінкових шаблонів лекція виділяє «Спостерігач» і «Стратегію», серед твірних — «Одинак» і «Фабрику», серед структурних архітектурних — MVC і багаторівневу архітектуру. Принципи SOLID (єдиної відповідальності, відкритості/закритості, підстановки Лісков, розділення інтерфейсів, інверсії залежностей) роблять систему легкою для підтримки, розширення та масштабування [7].

3. ВИБІР РІВНІВ РЕАЛІЗАЦІЇ ПРОГРАМНОГО ПРОДУКТУ

3.1. Вихідні вимоги до архітектури

Вибір рівнів і стилю ґрунтується на специфікації вимог практичної роботи № 1 [2]: 22 функціональні і 16 нефункціональних вимог. Для архітектури визначальними є нефункціональні вимоги, наведені в табл. 2.2: саме вони відрізняють «Щит-Лінк» від типових IoT-систем моніторингу (теплиці, клініки, насосні станції [5; 6]), де допустимі затримки в секунди-хвилини й робота лише в хмарі.

Таблиця 2.2 — Нефункціональні вимоги, що визначають архітектуру (з ПР1)

Код	Вимога	Критерій	Наслідок для архітектури
NFR-01	Затримка «сенсор — екран АРМ»	≤ 2 с (95-й процентиль)	Потокова обробка подій, доставка приростів через WebSocket
NFR-02	Доставка бойового завдання на планшет	≤ 1 с (95-й процентиль)	Окремий клас пріоритету для завдань у всіх каналах
NFR-03	Пікове навантаження прийому даних	≥ 20 000 повідомлень/с; ≥ 500 треків	Розподілена шина подій, масштабовані сервіси
NFR-05	Доступність ядра платформи	≥ 99,95 % на місяць	Два центри обробки даних, реплікація
NFR-06	Автономність шлюзу; відновлення ядра	≥ 4 год офлайн; RPO ≤ 5 с, RTO ≤ 60 с	Обчислення на периферійному шлюзі, буферизація
NFR-07	Взаємна автентифікація пристроїв і сервісів, шифрування	mTLS, TLS 1.3, AES-256	Сертифікат X.509 для кожного пристрою, VPN
NFR-15	Відповідність вимогам захисту інформації	Атестат відповідності КСЗІ	Власні сервери замість публічної хмари

3.2. Вибір рівнів реалізації

За основу взято чотирирівневу модель IoT-рішення (підрозділ 2.1). Для кожного рівня визначено, чи реалізується він у продукті, у якій формі та чим це зумовлено (табл. 2.3). Головна відмінність від еталонної моделі [4] — частина обробки даних перенесена на **периферійні шлюзи** мережевого рівня, а рівень обробки даних розміщено не в публічній хмарі, а в **двох власних**

центрах обробки даних (ЦОД) — через вимоги автономності (NFR-06) та захисту інформації (NFR-15).

Таблиця 2.3 — Обрані рівні реалізації IoT-платформи «Щит-Лінк»

Рівень	Реалізація в продукті	Обґрунтування
Рівень датчиків (сприйняття)	Власні пристрої не розробляються; інтегруються акустичні вузли, РЛС, оптико-електронні пости, наземні станції управління (НСУ) перехоплювачів	Обладнання постачається сторонніми виробниками; продукт є програмною платформою (FR-01, FR-02)
Мережевий рівень	Периферійний шлюз на кожній позиції (власне ПЗ), багатоканальний зв’язок (mesh-радіо, LTE, супутник), VPN	Різнорідні протоколи пристроїв, подавлення зв’язку, автономність ≥ 4 год (FR-03, NFR-06)
Рівень обробки даних	IoT-ядро у двох ЦОД: шина подій, мікросервіси, PostgreSQL + PostGIS, TimescaleDB	Обробка в реальному часі, доступність 99,95 %, захист інформації (NFR-01, NFR-05, NFR-15)
Рівень застосунків	Веб-АРМ командного пункту і служб, мобільний застосунок планшета, інтеграційні інтерфейси	Три форми продукту з ПР1: веб-система, мобільний застосунок, окремий додаток шлюзу

Отже, у продукті реалізуються всі чотири рівні, але з різною глибиною: рівень датчиків — лише як інтеграція (адаптери протоколів), мережевий рівень і рівень обробки даних — як власне програмне забезпечення, рівень застосунків — як два клієнтські застосунки та інтеграційні інтерфейси.

3.3. Вибір архітектурного стилю

Стилі лекції [7] порівняно за вимогами табл. 2.2 (табл. 2.4).

Таблиця 2.4 — Порівняння архітектурних стилів для IoT-ядра

Стиль	Відповідність вимогам	Обмеження	Рішення
Монолітна	Простота розробки й розгортання	Неможливо масштабувати лише сервіси реального часу; збій модуля зупиняє все ядро	Відхилено
Мікросервісна	Незалежне масштабування злиття треків і телеметрії; ізоляція збоїв; оновлення без зупинки	Складність координації та тестування	Обрано для сервісів ядра
SOA	Стандартизовані інтерфейси для зовнішніх систем	Централізована шина ESB — вузьке місце для 20 000 повідомлень/с	Ідеї взято для інтеграції
EDA	Пристрої генерують потоки подій; асинхронна обробка в реальному часі	Складність відлагодження, порядок подій	Обрано як основу взаємодії

Обрано **гібридний стиль**: подієво-орієнтована архітектура (EDA) як основа взаємодії — виявлення, треки, стани місій і телеметрія передаються як події через шину подій; сервіси ядра реалізовано як **мікросервіси**, кожен з яких є видавцем і/або підписником подій; взаємодія із зовнішніми системами ведеться через стандартизовані інтерфейси (принцип SOA) інтеграційного адаптера. Недоліки EDA компенсуються ключами впорядкування подій за треком і місією та наскрізними ідентифікаторами для трасування подій під час відлагодження.

4. АРХІТЕКТУРА ІОТ-ЕКОСИСТЕМИ «ЩИТ-ЛІНК»

4.1. Загальна схема архітектури

Архітектуру IoT-рішення побудовано за чотирирівневою моделлю у формі, прийнятій у публікаціях кафедри [4; 6]: колонки «пристрої та об’єкти — зв’язок — обробка даних — інтерфейси застосувань» відповідають рівням датчиків, мережевому, обробки даних і застосунків (рис. 2.2).

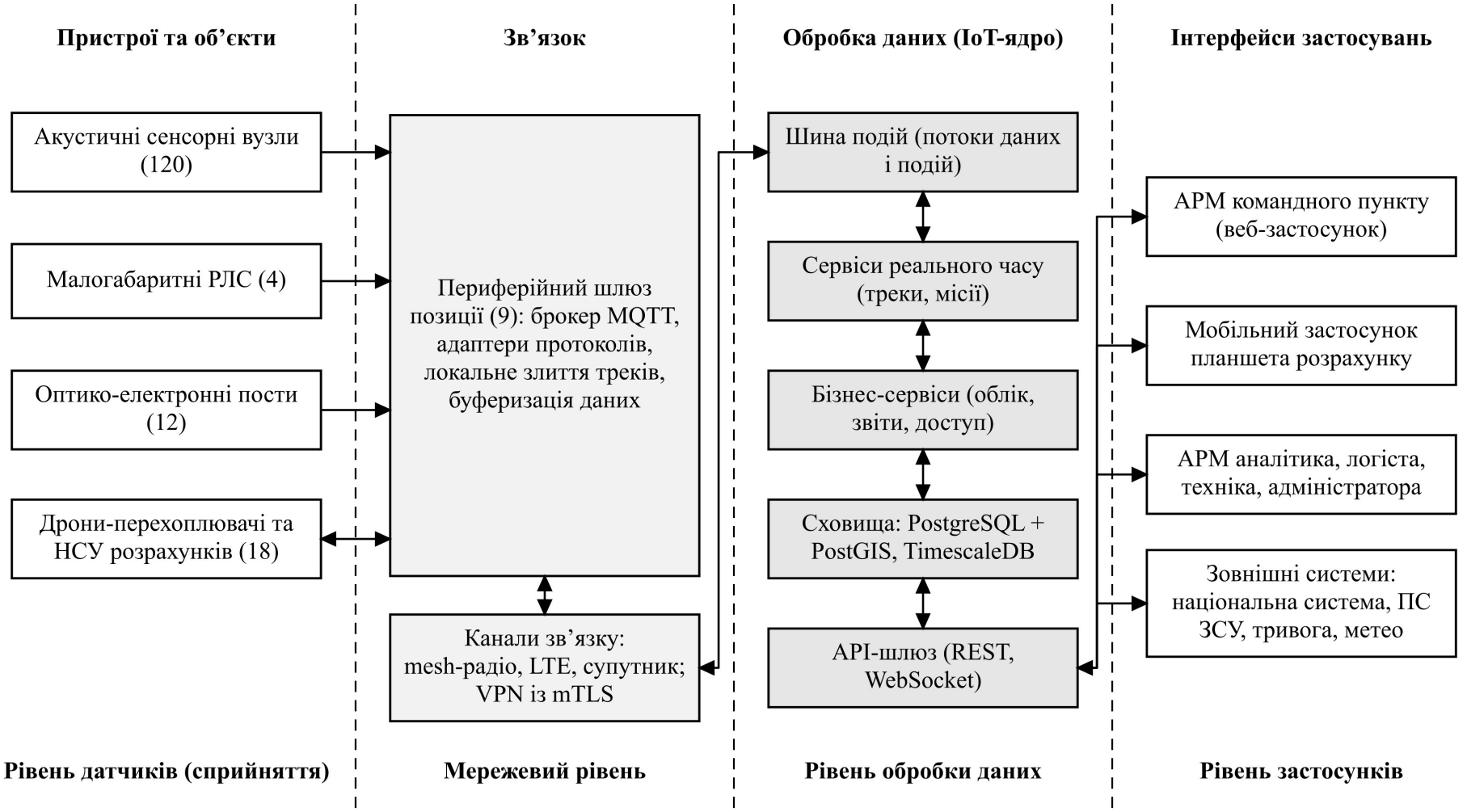


Рис. 2.2 — Архітектура IoT-екосистеми «Щит-Лінк»

Дані рухаються зліва направо: пристрої сектора передають виявлення й телеметрію на периферійний шлюз своєї позиції; шлюз перевіряє й нормалізує дані та через захищені канали передає їх до шини подій ядра; сервіси реального часу формують повітряну обстановку й керують місіями, бізнес-сервіси ведуть облік і звітність, сховища зберігають оперативні дані й часові ряди; API-шлюз надає дані застосункам і зовнішнім системам. Команди (призначення, санкції, завдання) рухаються у зворотному напрямку — від застосунків через ядро та шлюз до НСУ розрахунку, тому зв’язки між рівнями двобічні.

4.2. Рівень датчиків (сприйняття)

Рівень датчиків утворюють пристрої, що спостерігають повітряний простір, і пристрої розрахунків (табл. 2.5). Платформа не змінює їх вбудоване програмне забезпечення: взаємодія ведеться лише через стандартні протоколи, а кожен пристрій має власний сертифікат для автентифікації на шлюзі.

Таблиця 2.5 — Пристрої рівня датчиків

Пристрій	Дані	Частота	Протокол	К-сть
Акустичний сенсорний вузол	Пеленг, клас сигнатури, достовірність, стан вузла	1–5 повідомлень /с	MQTT 5 (TLS)	120
Малогабаритна РЛС	Відмітки й треки цілей: координати, висота, швидкість	1 Гц	ASTERIX CAT048/062	4
Оптико-електронний пост	Відеопотік, підтвердження виявлень	За запитом	RTSP, MQTT	12
НСУ розрахунку з перехоплювачем	Телеметрія: позиція, заряд АКБ, якість каналу, режим польоту	≥ 5 Гц у польоті	MAVLink 2 з підписом	18

4.3. Мережевий рівень

Мережевий рівень складається з периферійних шлюзів на дев’яти позиціях і каналів зв’язку між шлюзами та ядром (табл. 2.6). Шлюз (рис. 2.3) виконує функції, які в еталонній моделі належать IoT-шлюзу [5], і додатково — обчислення на периферії:

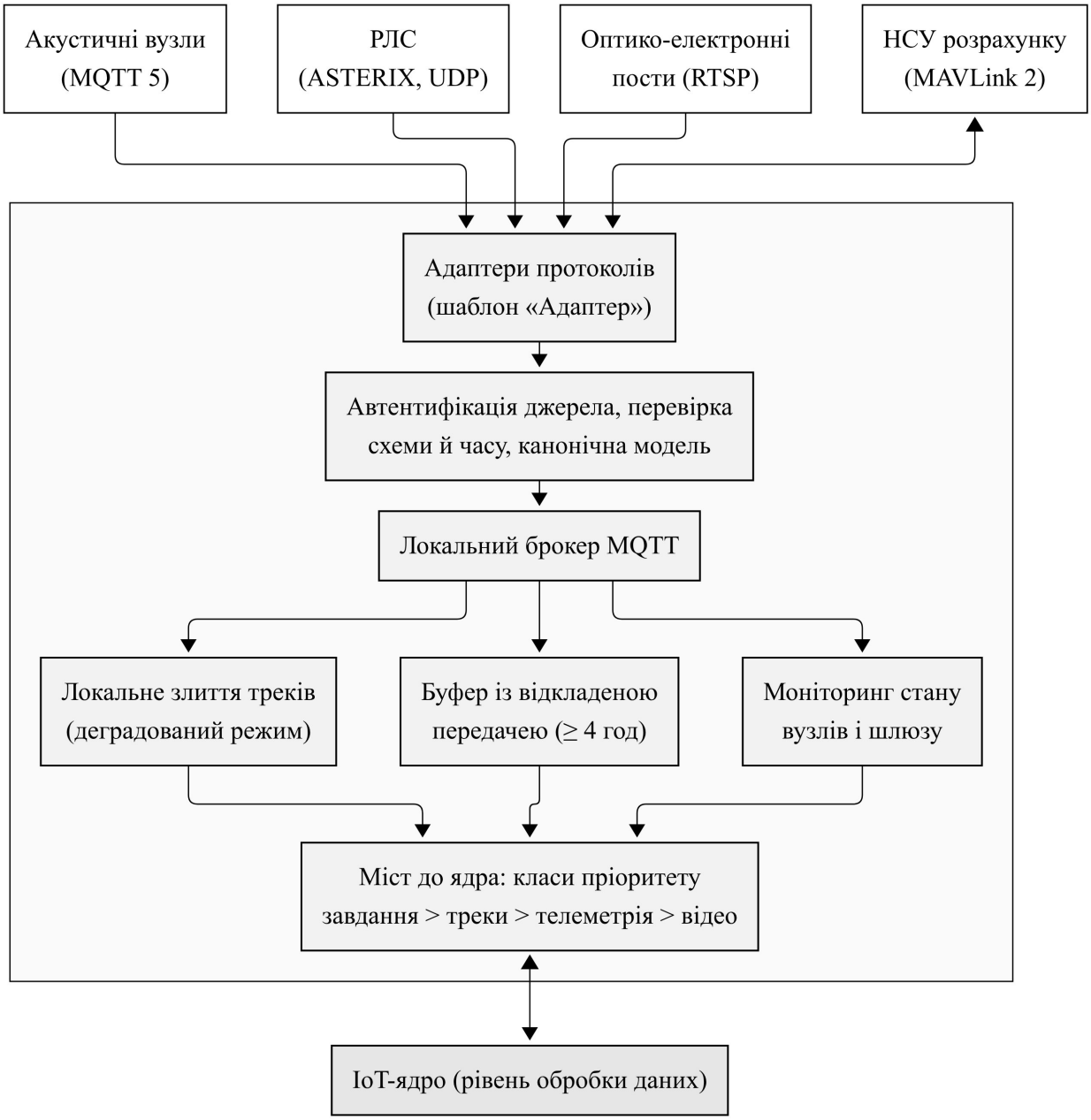


Рис. 2.3 — Структура периферійного шлюзу позиції

– **адаптери протоколів** перетворюють MQTT, ASTERIX, RTSP і MAVLink на єдину канонічну модель даних (шаблон «Адаптер»);

– **перевірка даних** — автентифікація джерела за сертифікатом, контроль схеми й часової мітки, переведення в систему координат WGS-84 і час UTC;

– **локальний брокер MQTT** розподіляє дані між внутрішніми компонентами шлюзу;

– **локальне злиття треків** у деградованому режимі дозволяє позиції працювати при втраті зв’язку з ядром;

– **буфер із відкладеною передачею** зберігає дані до 4 годин і передає їх після відновлення зв’язку з первинними мітками часу;

– **міст до ядра** передає дані за класами пріоритету: завдання > треки > телеметрія > відео.

Таблиця 2.6 — Канали та протоколи мережевого рівня

Зв’язок	Протокол	Канал	Вимога
Пристрої → шлюз	MQTT 5, ASTERIX, MAVLink 2	Локальна мережа позиції, радіоканал	Сертифікат кожного пристрою
Шлюз ↔ планшет	MQTT / WebSocket через TLS	Wi-Fi позиції	Робота без ядра
Шлюз ↔ ядро	gRPC або міст MQTT через VPN із mTLS	Mesh-радіо, LTE, супутник (резервування)	Класи пріоритету трафіку
Ядро ↔ застосунки	HTTPS (REST), WebSocket	Захищена мережа центру	Прирости даних
Ядро ↔ зовнішні системи	REST API, Cursor-on-Target	Захищені канали обміну	Звіт ≤ 15 хв

4.4. Рівень обробки даних

Рівень обробки даних — IoT-ядро, побудоване за обраним гібридним стилем (рис. 2.4). Точками входу є шлюз прийому даних пристроїв, інтеграційний адаптер і API-шлюз; усі вони взаємодіють із **шиною подій**, яка виконує роль брокера подій EDA [7]. Дев’ять мікросервісів є видавцями й підписниками подій; сервіси реального часу (на схемі виділено сірим) масштабуються незалежно від бізнес-сервісів. Кожен сервіс має власні дані: оперативні та просторові — у PostgreSQL з розширенням PostGIS, часові ряди телеметрії та показників — у TimescaleDB.

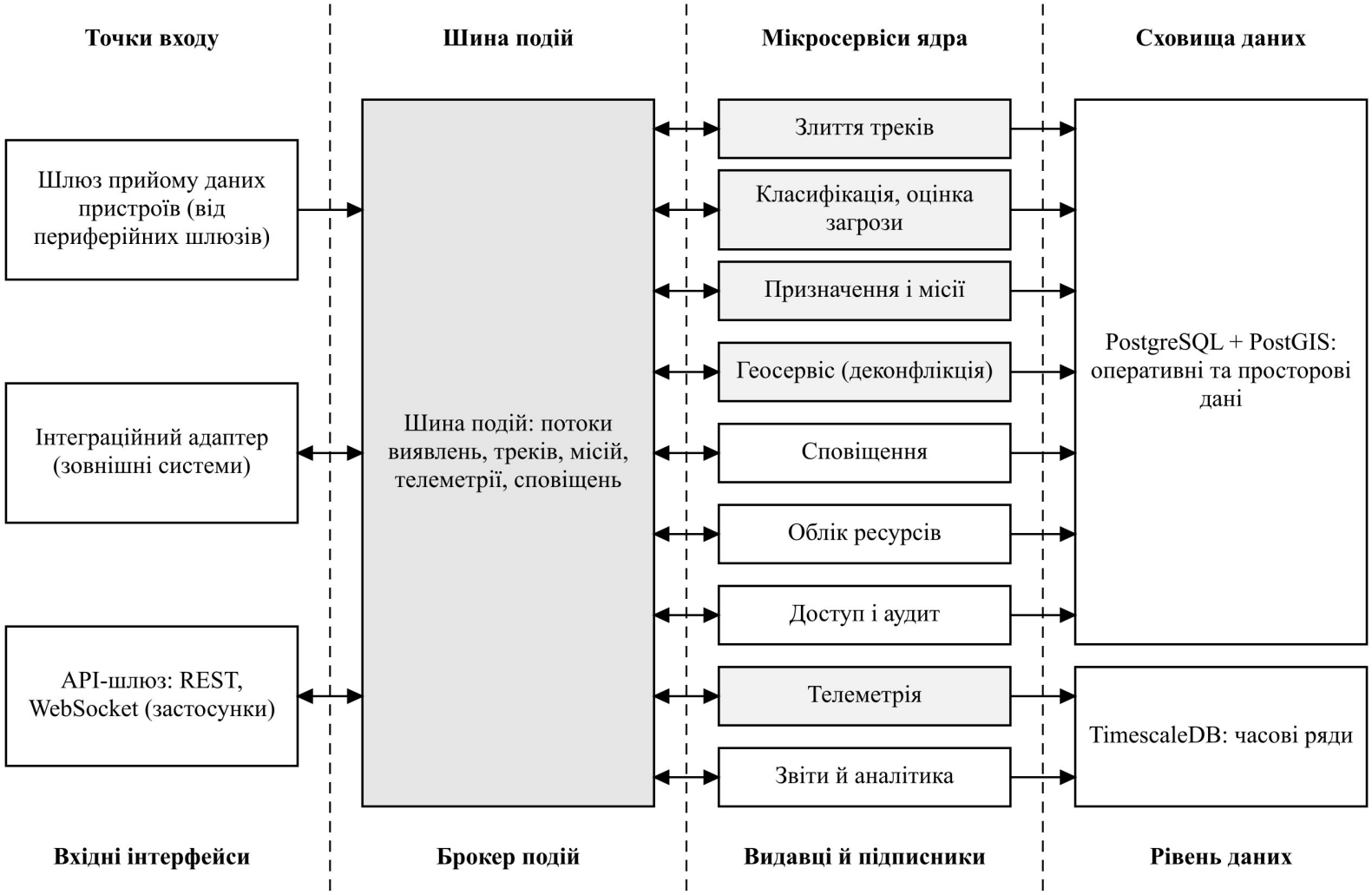


Рис. 2.4 — Компоненти рівня обробки даних (IoT-ядра)

Відповідальність кожного мікросервісу, спосіб його взаємодії та функціональні вимоги ПР1, які він реалізує, наведено в табл. 2.7.

Таблиця 2.7 — Мікросервіси IoT-ядра

Сервіс	Відповідальність	Взаємодія	FR
Злиття треків	Асоціація виявлень із треками, тріангуляція пеленгів, життєвий цикл треку, прогноз траєкторії	Підписник «виявлення», видавець «трек оновлено»	FR-04
Класифікація, оцінка загрози	Підказка типу цілі, звіряння з планами польотів, оцінка загрози та пріоритет	Підписник «трек оновлено», видавець «трек класифіковано»	FR-05 FR-06
Призначення і місії	Перелік кандидатів, стани місії, правило двох осіб, ескалація без підтвердження	Команди через API; видавець «місія змінилася»	FR-08 … FR-12
Геосервіс (деконфлікція)	Сектори, зони заборони ураження, коридори дружньої авіації, просторові перевірки	Синхронні запити від сервісу призначення	FR-07
Сповіщення	Сповіщення за подіями, рівні важливості, ескалація непідтверджених	Підписник багатьох потоків	FR-16
Облік ресурсів	Готовність розрахунків, перехоплювачів, АКБ; обслуговування; запаси й заявки	Команди через API; видавець «ресурс змінився»	FR-14 FR-15
Доступ і аудит	Користувачі, ролі, двофакторна автентифікація, журнал аудиту з ланцюжком хешів	Запити від API-шлюзу; підписник усіх команд	FR-19 FR-20
Телеметрія	Прийом і архівування телеметрії перехоплювачів, відтворення місій	Підписник «телеметрія»	FR-12 FR-17
Звіти й аналітика	Звіти про перехоплення, показники KPI, експорт PDF / XLSX	Підписник «місія закрита»; запити API	FR-13 FR-17 FR-21

Шина подій зберігає потоки подій у вигляді журналу, тому новий сервіс може під’єднатися до потоку без змін у видавцях (принцип відкритості/закритості), а після збою сервіс продовжує обробку з місця зупинки. Для швидкого відображення обстановки API-шлюз тримає в пам’яті актуальний стан треків і надсилає застосункам лише прирости.

4.5. Рівень застосунків

Рівень застосунків реалізує інтерфейси для всіх акторів ПР1 [2] (табл. 2.8). Застосунки не звертаються до сервісів напряму: усі запити проходять через API-шлюз, який перевіряє права доступу.

Таблиця 2.8 — Застосунки та інтерфейси рівня застосунків

Застосунок	Користувачі	Технологія	Основні функції
АРМ командного пункту	Оперативний черговий, офіцер бойового управління, аналітик обстановки	Веб-застосунок (TypeScript), карта з умовними знаками APP-6	Повітряна обстановка, класифікація, призначення, санкціонування, супровід місій
Мобільний застосунок планшета	Командир розрахунку, оператор перехоплювача	Android 12+	Прийняття завдання, цілевказання, доповідь про результат; робота через шлюз без ядра
АРМ служб забезпечення	Технік, логіст, аналітик ефективності, адміністратор, офіцер кібербезпеки	Веб-застосунок	Облік ресурсів, звіти й KPI, користувачі й ролі, журнал аудиту
Інтеграційні інтерфейси	Національна система, Повітряні сили ЗСУ, система тривоги, метеосервіс	REST API, Cursor-on-Target	Обмін обстановкою, плани польотів, сповіщення, метеодані

4.6. Фізичне розміщення компонентів

Рівні архітектури розміщено на фізичних вузлах, наведених у табл. 2.9. Сервіси ядра й шлюзу постачаються як контейнери, що забезпечує однакову конфігурацію в усіх середовищах, як рекомендує модель DevOps [8].

Таблиця 2.9 — Фізичне розміщення компонентів

Вузол	Розташування	Програмні компоненти	К-сть
Периферійний шлюз	Позиція розрахунку	Адаптери, брокер MQTT, локальне злиття, буфер, міст до ядра (контейнери, Linux)	9 + 2
Кластер ядра (основний)	ЦОД 1	Шина подій, мікросервіси, API-шлюз, інтеграційний адаптер	1
Кластер ядра (резервний)	ЦОД 2	Ті самі сервіси в режимі гарячого резерву; репліка баз даних	1
АРМ	Командний пункт, служби	Браузер із веб-застосунком	8 + 6
Планшет	Розрахунок	Мобільний застосунок	18 + 4

4.7. Наскрізний потік даних і бюджет затримок

Взаємодію рівнів у ключовому сценарії «виявлення — обстановка — завдання розрахунку» показано на діаграмі послідовності (рис. 2.5). Від виявлення до появи приросту обстановки на АРМ має минути не більше 2 с (NFR-01), від санкції до отримання завдання планшетом — не більше 1 с (NFR-02). Розподіл цих бюджетів між рівнями наведено в табл. 2.10.

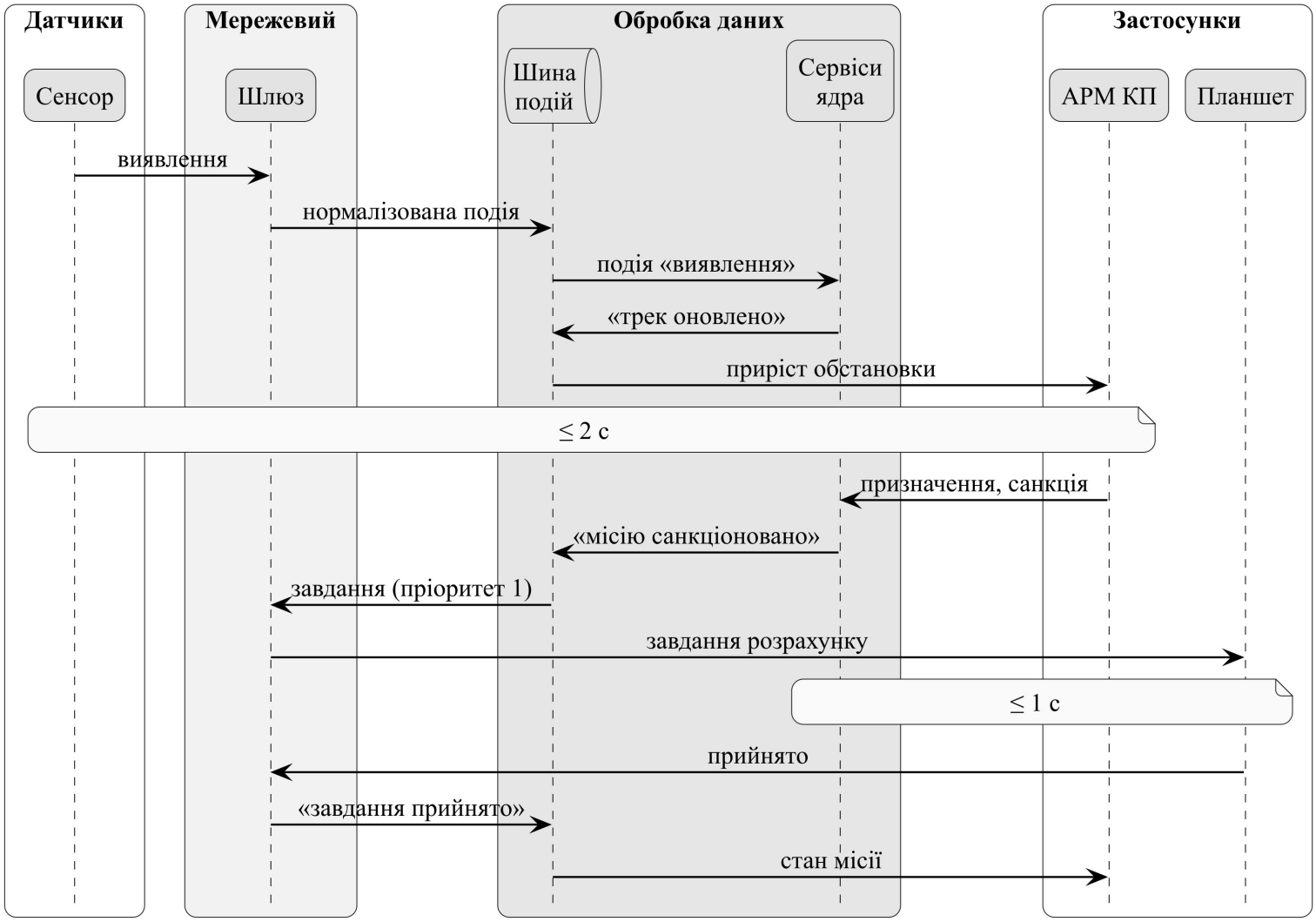


Рис. 2.5 — Взаємодія рівнів у сценарії «виявлення — завдання розрахунку»

Таблиця 2.10 — Бюджет затримок за рівнями архітектури

Етап	Рівень	Бюджет, мс
Передача виявлення від сенсора до шлюзу	Датчиків → мережевий	≤ 300
Перевірка, нормалізація, публікація на шлюзі	Мережевий	≤ 100
Передача шлюз → шина подій	Мережевий	≤ 400
Злиття треків і публікація оновлення	Обробки даних	≤ 500
Доставка приросту на АРМ і відображення	Обробки даних → застосунків	≤ 700
Разом «сенсор — екран» (NFR-01)	—	≤ 2000
Санкція → подія → шлюз → планшет (NFR-02)	Усі рівні	≤ 1000

4.8. Безпека та надійність

Захист реалізовано на кожному рівні архітектури (табл. 2.11), що відповідає нефункціональним вимогам NFR-07…NFR-09 і NFR-15 [2].

Таблиця 2.11 — Заходи безпеки за рівнями архітектури

Рівень	Загрози	Заходи
Датчиків	Підміна пристрою, хибні дані	Сертифікат X.509 кожного пристрою, підпис MAVLink 2, карантин вузла при різкій зміні координат
Мережевий	Перехоплення й підміна трафіку, подавлення зв’язку	VPN із взаємною автентифікацією (mTLS), TLS 1.3, резервні канали, обмеження частоти повідомлень
Обробки даних	Несанкціонований доступ, зміна даних	Шифрування AES-256 у сховищах, права на рівні сервісів, журнал аудиту з ланцюжком хешів
Застосунків	Викрадення облікових даних, перевищення повноважень	Двофакторна автентифікація, рольовий доступ з обмеженням за сектором, блокування сеансу через 15 хв

Надійність забезпечують три механізми: резервування ядра у двох ЦОД з реплікацією потоків подій (відновлення ≤ 60 с, втрата даних ≤ 5 с — NFR-06); автономна робота периферійних шлюзів щонайменше 4 год із локальним злиттям треків і подальшим узгодженням станів, у якому головним джерелом для станів місій є ядро; пріоритизація трафіку, що за перевантаження каналів першою обмежує передачу відео.

4.9. Шаблони проектування та принципи SOLID в архітектурі

Шаблони проектування та принципи SOLID лекції [7] застосовано в архітектурі, як показано в табл. 2.12.

Таблиця 2.12 — Застосування шаблонів проектування та принципів SOLID

Шаблон / принцип	**Де застосовано**	**Ефект**
Адаптер	Адаптери протоколів шлюзу, інтеграційний адаптер ядра	Нові типи пристроїв і систем без зміни ядра
Фасад	API-шлюз для всіх застосунків	Простий інтерфейс до дев’яти сервісів
Проксі	Кеш стану обстановки в API-шлюзі; перевірка прав перед викликом сервісу	Швидкодія та контроль доступу
Спостерігач	Підписка сервісів і застосунків на потоки подій	Відокремленість видавців і підписників
Стратегія	Алгоритми ранжування розрахунків-кандидатів і класифікації	Заміна алгоритму без зміни сервісу
Фабрика	Створення адаптера за типом пристрою під час реєстрації вузла	Єдина точка створення адаптерів
MVC, багаторівнева архітектура	Веб- і мобільний застосунки; рівні «інтерфейс — логіка — дані» в сервісах	Окреме тестування рівнів
SOLID: S, O, D	Один сервіс — одна відповідальність; нові підписники без змін видавців; сервіси залежать від інтерфейсів сховищ	Легкість супроводу та заміни технологій

4.10. Відповідність архітектури вимогам

Трасування нефункціональних вимог ПР1 до архітектурних рішень наведено в табл. 2.13; кожна вимога, що визначає архітектуру (табл. 2.2), має щонайменше одне рішення.

Таблиця 2.13 — Трасування вимог до архітектурних рішень

Вимога	Архітектурне рішення	Рівень
NFR-01	Подієва обробка, прирости через WebSocket, кеш обстановки в API-шлюзі	Обробки даних, застосунків
NFR-02	Найвищий клас пріоритету для завдань у мосту шлюзу та шині подій	Мережевий, обробки даних
NFR-03, NFR-04	Розподілена шина подій, незалежне масштабування мікросервісів реального часу	Обробки даних
NFR-05	Два ЦОД, гарячий резерв, реплікація	Обробки даних
NFR-06	Буфер і локальне злиття треків на периферійному шлюзі	Мережевий
NFR-07, NFR-08	Сертифікати пристроїв, mTLS, двофакторна автентифікація, рольовий доступ	Усі рівні
NFR-09	Сервіс доступу й аудиту — підписник усіх команд, журнал із ланцюжком хешів	Обробки даних
NFR-12	Мікросервіси, контейнери, конвеєр CI/CD	Обробки даних
NFR-15	Власні ЦОД замість публічної хмари, КСЗІ	Обробки даних

ВИСНОВКИ

У практичній роботі побудовано та описано архітектуру IoT-рішення для програмного продукту IoT-платформа «Щит-Лінк» умовного РЦПДО «Щит-Північ».

1) За основу взято чотирирівневу модель IoT-рішення з публікацій кафедри: рівень датчиків (сприйняття), мережевий рівень, рівень обробки даних і рівень застосунків. Усі чотири рівні реалізовано в продукті: рівень датчиків — як інтеграція 154 пристроїв чотирьох типів через стандартні протоколи, мережевий рівень — як власне ПЗ дев'яти периферійних шлюзів і багатоканальний захищений зв'язок, рівень обробки даних — як IoT-ядро у двох власних ЦОД, рівень застосунків — як веб-АРМ, мобільний застосунок і інтеграційні інтерфейси.

2) Порівняно чотири архітектурні стилі лекції; обрано гібридний стиль: подієво-орієнтована архітектура з шиною подій як основа взаємодії та 9 мікросервісів ядра, а також стандартизовані інтерфейси інтеграції за принципом SOA. Монолітну архітектуру відхилено через неможливість незалежно масштабувати сервіси реального часу.

3) Відмінності від еталонної моделі обґрунтовано вимогами ПР1: обчислення на периферійних шлюзах забезпечують автономність позиції щонайменше 4 год, а власні ЦОД замість публічної хмари — вимоги захисту інформації.

4) Описано кожен рівень, фізичне розміщення компонентів, наскрізний потік даних з бюджетом затримок (≤ 2 с до екрана, ≤ 1 с до планшета), заходи безпеки на кожному рівні, застосування шести шаблонів проектування й принципів SOLID; усі нефункціональні вимоги, що визначають архітектуру, трасовано до архітектурних рішень.

Побудована архітектура є основою для опису існуючих програмних та апаратних рішень (практична робота № 3) і детального проектування системи засобами BPMN, UML та IDEF (практична робота № 4).

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Практична робота 2. Архітектура IoT екосистеми : завдання з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026.

2. Кравченко П. Збір та документування вимог для програмного продукту : звіт з практичної роботи № 1 з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026. 42 с.

3. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ : ДП «УкрНДНЦ», 2016. 26 с.

4. Zinko I., Kravchenko O., Syvoglaz D. Designing an Internet of Things solution for monitoring vital signs. Technology Audit and Production Reserves. 2024. No. 2 (2 (76)). DOI: 10.15587/2706-5448.2024.300745.

5. Васюта А., Кравченко О. IoT система керування ветеринарною клінікою. Інформаційні системи та технології: результати і перспективи : матеріали 2-ї Міжнар. наук.-практ. конф. (IST 2025), м. Київ, 5 берез. 2025 р. Київ : ФІТ КНУТШ, 2025. С. 36–38.

6. Легойда Ю. В., Кравченко О. В. Автоматизована IoT система управління насосної станції водопостачання селища Теплик. Таврійський науковий вісник. Серія: Технічні науки. 2024. № 2. С. 73–87. DOI: 10.32782/tnv-tech.2024.2.7.

7. Кравченко О. Проектування програмних систем : лекція 5 з дисципліни «Технології створення програмних продуктів». Київ : КНУ імені Тараса Шевченка, 2026. 43 слайди.

8. Життєвий цикл та моделі життєвого циклу програмного продукту : лекція 1 з дисципліни «Технології створення програмних продуктів» / викл. О. В. Кравченко. Київ : КНУ імені Тараса Шевченка, 2026. 40 слайдів.